

Informe Trabajo Práctico 2

Autores: Catalina Chab, Catalina Brusco y Juan Ignacio Castore

Decisiones de diseño

Para este trabajo práctico decidimos aplicar **aprendizaje por refuerzo** al clásico juego *Connect4*. Nuestro objetivo fue entrenar un agente que pueda aprender a jugar de manera competitiva y, al menos, superar a un jugador que elige de forma aleatoria.

El primer paso fue definir el ambiente de juego. Representamos el tablero como una matriz de 6 filas por 7 columnas (esto se puede modificar a gusto). Cada acción corresponde a elegir una columna válida para colocar una ficha. La condición de finalización se da cuando algún jugador logra alinear cuatro fichas o cuando el tablero se llena. Las recompensas se definieron de manera simple: +1 cuando el agente gana, -1 cuando pierde, y 0 en el resto de las jugadas (incluyendo empates).

Una decisión importante fue elegir el algoritmo de aprendizaje. Optamos por Deep Q-Learning (DQN), ya que lo vimos en clase, es un estándar en este tipo de problemas y permite aprovechar una red neuronal para estimar los valores de cada jugada.

Entrenamiento del Agente

Para entrenar al agente implementamos un ciclo clásico de episodios: el agente juega partidas completas contra un oponente, guarda sus experiencias y ajusta la red neuronal con lo aprendido.

- **Estados:** el tablero en su configuración actual, el current player (1 o 2), un flag de si el estado es terminal o no, si ya se definió el winner y el número de jugada actual.
- **Acciones:** columnas donde se puede jugar.
- **Recompensas:** victoria, derrota o empate.

Experimentamos entrenarlo con el **RandomAgent** (que juega siempre al azar) y con el **DefenderAgent**, que al menos bloquea jugadas obvias de victoria. En esencia, entrenamos nuestro agente **sólo con el DefenderAgent** porque el RandomAgent (puramente aleatorio) proporcionaba una señal de aprendizaje de baja calidad, resultando en un rendimiento inferior durante las pruebas.

Durante el entrenamiento usamos parámetros de exploración (epsilon-greedy) que arrancan altos para permitir probar jugadas diversas, y luego va bajando de manera exponencial (empieza en 1 y se va multiplicando por 0.995 sucesivamente, hasta llegar como mínimo =0.1) para que el agente se enfoque en lo aprendido. También experimentamos con distintas arquitecturas de red, variamos epochs, cantidad de capas y probamos una red residual para mejorar la estabilidad y rendimiento de la misma.

Nuestro agente final (trained_model_vs_DefenderAgent_1000_0.99_1.0_0.1_0.9950.001_128_1000_100.pth) usó los siguientes hiperparámetros:

Parámetro	Valor	Descripción
Episodios	1000	Número de iteraciones de entrenamiento.
Oponente	DefenderAgent	Agente utilizado para el entrenamiento, configurado para

		bloquear victorias.
Gamma (γ)	0.99 ▾	Factor de descuento, indicando una alta valoración de recompensas futuras.
Epsilon inicial	1.0 ▾	Probabilidad inicial del 100% para la exploración.
Epsilon mínimo	0.1 ▾	Probabilidad mínima de exploración, establecida en 10% para asegurar una exploración continua.
Epsilon decay	0.995 ▾	Decaimiento exponencial suave de la probabilidad de exploración.
Learning rate (α)	0.001 ▾	Tasa de aprendizaje conservadora, optimizada mediante Adam.
Batch size	128 ▾	Tamaño del mini-lote para las actualizaciones de la red.
Memory size	1000 ▾	Capacidad del búfer de repetición de experiencias.
Target update	100 ▾	Frecuencia de actualización de la red objetivo.

Resultados

El desempeño del agente fue claro:

- **Contra el RandomAgent**, nuestro modelo entrenado logró superar la gran mayoría de las partidas, el 75%aprox. **Contra el DefenderAgent**, los resultados fueron más desafiantes(21% aprox.), pero se observó un progreso a medida que ajustamos la arquitectura y entrenamos más episodios.
- En general, el agente mostraba conductas cada vez menos aleatorias y más “intencionales”, como priorizar jugadas centrales o bloquear al rival en algunos casos.

Esto muestra que el aprendizaje por refuerzo fue capaz de captar patrones básicos del juego y usarlos para mejorar el rendimiento.

Conclusión

El desarrollo nos permitió experimentar de primera mano las dificultades de diseñar un **ambiente de RL**: cómo definir recompensas que guíen bien al agente, cómo balancear exploración y explotación, y qué oponentes elegir para que el entrenamiento sea desafiante pero posible. Entre los principales aprendizajes destacamos:

- La importancia de elegir buenos **hiperparámetros** (la arquitectura de la red y la memoria de experiencias).
- La influencia del oponente en el proceso de entrenamiento. Cuando lo entrenamos con el Random, fue peor el agente resultante que cuando lo entrenamos con el Defender.

Como mejora futura, sería interesante entrenar más episodios contra agentes más sofisticados y quizás diseñar recompensas intermedias que premien jugadas estratégicas, no solo la victoria final.